

LAPORAN PRAKTIKUM
PEMOGRAMAN BERORIENTASI OBJEK
“Keamanan & Integritas Data (Enkapsulasi)”



disusun Oleh:
NAIRA RAMADHANI HALIL
2511533027

Dosen Pengampu:
NURFIAH, S.ST., M.Kom.

Asisten Praktikum:
Siti Fadhilah Rahmi

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan salah satu konsep penting dalam pengembangan program karena program dapat dibuat berdasarkan objek yang memiliki data dan perilaku. Dalam penerapannya, setiap objek dapat memiliki atribut dan method yang digunakan untuk mengelola data serta menjalankan proses tertentu.

Pada praktikum ini, konsep PBO diterapkan melalui studi kasus program Mini Bank. Program digunakan untuk melakukan beberapa proses perbankan sederhana seperti membuka rekening, setor tunai, tarik tunai, melihat informasi rekening, mengganti akun, serta melihat total transaksi. Program juga menggunakan beberapa class yang saling berhubungan, yaitu class Rekening dan class Transaksi.

Pada pengembangan program ini, ditambahkan fitur keamanan menggunakan PIN pada setiap rekening. Saat membuka rekening, pengguna diminta memasukkan PIN berupa 6 digit yang kemudian disimpan pada objek Rekening. PIN tersebut digunakan sebagai proses autentikasi sebelum pengguna dapat melakukan penarikan tunai dan melihat riwayat transaksi.

Penerapan autentikasi PIN dilakukan dengan menggunakan method otentikasi() pada class Rekening. Sistem akan membandingkan PIN yang dimasukkan pengguna dengan PIN yang tersimpan. Jika PIN benar, proses dapat dilanjutkan, sedangkan jika PIN salah, sistem akan menolak akses dan menampilkan pesan bahwa PIN yang dimasukkan salah.

Melalui praktikum ini, mahasiswa dapat memahami penerapan konsep enkapsulasi, constructor, method, ArrayList, asosiasi antar objek, serta autentikasi sederhana pada program Java. Dengan demikian, program Mini Bank tidak hanya dapat mengelola transaksi, tetapi juga memiliki mekanisme keamanan untuk membatasi akses terhadap transaksi tertentu.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami penerapan konsep Pemrograman Berorientasi Objek dalam program Java.
2. Memahami penggunaan class, object, constructor, atribut, dan method.
3. Menerapkan konsep enkapsulasi dengan menggunakan atribut private.
4. Menerapkan asosiasi one-to-many antara class Rekening dan Transaksi.
5. Menggunakan ArrayList untuk menyimpan data rekening dan riwayat transaksi.
6. Menerapkan PIN sebagai mekanisme autentikasi pada rekening.
7. Membatasi akses tarik tunai dan informasi transaksi berdasarkan validasi PIN.
8. Memahami penggunaan method otentikasi() untuk melakukan pengecekan PIN pengguna.

BAB II PEMBAHASAN

2.1 Rekening.java

```
package MiniBank.copy.copy;

import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;
    private String namaPemilik;
    private double saldo;
    private String pin; // Data sensitif

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat.");
        // Wajib menginstalikasi ArrayList didalam constructor agar tidak
NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }
```

```

//4. Method Otentikasi Internal (Validasi Enkapsulasi)
public boolean otentikasi(String inputPin) {
    return this.pin.equals(inputPin);
}

public void setorTunai(double nominal) {
    if (nominal > 0) {
        saldo += nominal;
        // Merekam riwayat (Pembuatan objek transaksi di dalam
method)
        String idTrx = "TRX-S" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Setor tunai Rp" + nominal+ " berhasil.
Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari
0!");
    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : " + saldo);
}

// Method tarik tunai
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println(
            "Transaksi Gagal: Minimal nominal penarikan Rp10000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;
        // Merekam riwayat penarikan
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Penarikan berhasil.");
        System.out.println("Jumlah penarikan: Rp" + nominal);
        System.out.println("Saldo sekarang: Rp" + saldo);
    }
}

public void cekMutasi() {
    if (riwayatTransaksi.isEmpty()) {

```

```

        System.out.println("Belum ada transaksi pada rekening
ini");
    } else {
        int mulai = Math.max(0, riwayatTransaksi.size() -
3);
        System.out.println("--- TRANSAKSI TERBARU ---");
        for (int i = mulai; i < riwayatTransaksi.size();
i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }
}

public void cetakTotalTransaksi() {

    double totalSetor = 0;
    double totalTarik = 0;

    for (Transaksi trx : riwayatTransaksi) {

        if (trx.getJenis().equals("Kredit")) {
            totalSetor += trx.getNominal();
        } else if (trx.getJenis().equals("Debit")) {
            totalTarik += trx.getNominal();
        }
    }

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

    System.out.println("--- TOTAL TRANSAKSI ---");
    System.out.println("Total Setor : " + rupiah.format(totalSetor));
    System.out.println("Total Tarik : " + rupiah.format(totalTarik));

    System.out.println("\n--- 3 TRANSAKSI TERBARU ---");

    int mulai = Math.max(0, riwayatTransaksi.size() - 3);

    for (int i = mulai; i < riwayatTransaksi.size(); i++) {
        riwayatTransaksi.get(i).cetakDetail();
    }

}
}

```

- **Atribut Private**

- private String nomorRekening;
- private String namaPemilik;
- private double saldo;
- private String pin;

Atribut tersebut digunakan untuk menyimpan nomor rekening, nama pemilik, saldo, dan PIN rekening. Penggunaan private merupakan penerapan enkapsulasi sehingga data tidak dapat diakses secara langsung dari class lain.

- **Constructor Rekening**

- `public Rekening(String nomor, String nama, double saldoAwal, String pinAwal)`

Constructor digunakan untuk memberikan nilai awal pada objek Rekening. Selain nomor rekening, nama pemilik, dan saldo awal, constructor sekarang menerima pinAwal sehingga PIN yang dimasukkan pengguna dapat disimpan pada rekening.

- **Validasi PIN**

- `if (pinAwal.matches("\\d{6}")) {
 this.pin = pinAwal;
} else {
 System.out.println("Peringatan: PIN harus 6 digit! Menggunakan PIN default 123456");
 this.pin = "123456";
}`

Bagian ini digunakan untuk memeriksa PIN yang dimasukkan pengguna. Kondisi `matches("\\d{6}")` memastikan PIN terdiri dari tepat 6 digit angka. Jika sesuai, PIN disimpan. Jika tidak sesuai, sistem menggunakan PIN default 123456.

- **Method Otentikasi**

- `public boolean otentikasi(String inputPin) {
 return this.pin.equals(inputPin);
}`

Method `otentikasi()` digunakan untuk membandingkan PIN yang dimasukkan pengguna dengan PIN yang tersimpan pada rekening. Method mengembalikan nilai `true` jika PIN sesuai dan `false` jika PIN berbeda.

- **Method Setor Tunai**

- `public void setorTunai(double nominal)`

Method ini digunakan untuk melakukan penyetoran uang ke rekening. Jika nominal lebih dari 0, saldo akan bertambah dan transaksi akan dicatat ke dalam riwayatTransaksi sebagai jenis Kredit.

- **Method Tarik Tunai**

- `public void tarikTunai(double nominal)`

Method ini digunakan untuk melakukan penarikan uang dari rekening. Sistem memeriksa minimal penarikan dan kecukupan saldo sebelum transaksi dilakukan. Jika berhasil, saldo dikurangi dan transaksi dicatat sebagai jenis Debit.

- **Penyimpanan Riwayat Transaksi**

- `ArrayList<Transaksi> riwayatTransaksi;`

Bagian ini menunjukkan hubungan antara objek Rekening dan objek Transaksi. Satu rekening dapat memiliki banyak transaksi sehingga menerapkan hubungan asosiasi one-to-many.

`this.riwayatTransaksi = new ArrayList<>();`

Digunakan untuk membuat objek ArrayList yang akan menyimpan kumpulan transaksi pada rekening.

- **Method Cetak Total Transaksi**

- `public void cetakTotalTransaksi()`

Method ini digunakan untuk menghitung total transaksi setor dan tarik berdasarkan riwayat transaksi yang tersimpan. Selain itu, method juga menampilkan tiga transaksi terbaru dari rekening aktif.

2.2 Main.java

```
package MiniBank.copy.copy;

import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
```

```

ArrayList<Rekening> daftarRekening = new ArrayList<>();
Rekening akunAktif = null;
boolean isRunning = true;
System.out.println("=== SISTEM PERBANKAN MINI ===");
while (isRunning) {

    System.out.println("\nMenu Utama");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Total Transaksi");
    System.out.println("7. Keluar");
    System.out.print("Pilih Menu: ");

    int pilihan = input.nextInt();
    input.nextLine();

    switch (pilihan) {
    case 1:
        System.out.print("Masukkan No Rekening: ");
        String no = input.nextLine();

        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();

        System.out.print("Masukkan Saldo Awal: ");
        double saldo = input.nextDouble();
        input.nextLine(); // membersihkan enter

        System.out.print("Masukkan PIN (6 digit): ");
        String pin = input.nextLine();

        Rekening rekeningBaru = new Rekening(no, nama, saldo,
pin);

        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;

        System.out.println("Rekening berhasil ditambahkan.");
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
    }
}

```

```

        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan PIN: ");
            String pinTarik = input.nextLine();

            // Verifikasi PIN sebelum tarik tunai
            if (akunAktif.otentikasi(pinTarik)) {
                System.out.print("Masukkan nominal tarik
tunai: ");

                double tarik = input.nextDouble();
                akunAktif.tarikTunai(tarik);
            } else {
                System.out.println(
                    "Akses Ditolak: PIN yang Anda masukkan
salah!"
                );
            }
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening yang
tersedia.");
        } else {
            System.out.print(
                "Masukkan nomor rekening: "
            );
            String nomorCari = input.nextLine();
            boolean ditemukan = false;
            for (Rekening rekening : daftarRekening) {
                if
(rekening.getNomorRekening().equals(nomorCari)) {
                    akunAktif = rekening;
                    ditemukan = true;
                    System.out.println("Akun berhasil diganti
ke " + rekening.getNamaPemilik());
                }
            }
        }
    }
}

```

```

        break;
    }
    }
    if (!ditemukan) {
        System.out.println("Rekening tidak
ditemukan.");
    }
    }
    break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan PIN: ");
            String pinMutasi = input.nextLine();

            // Verifikasi PIN sebelum cetak mutasi
            if (akunAktif.otentikasi(pinMutasi)) {
                akunAktif.cetakTotalTransaksi();
            } else {
                System.out.println(
                    "Akses Ditolak: PIN yang Anda masukkan
salah!"
                );
            }
        }
        break;

    case 7:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
}
input.close();
}
}

```

- **Membuat Daftar Rekening**
 - `ArrayList<Rekening> daftarRekening = new ArrayList<>();`
Digunakan untuk menyimpan beberapa objek Rekening yang dibuat selama program berjalan.
- **Menentukan Akun Aktif**
 - `Rekening akunAktif = null;`

Variabel `akunAktif` digunakan untuk menyimpan rekening yang sedang digunakan oleh pengguna.

- **Menu Buka Rekening**

- `System.out.print("Masukkan PIN (6 digit): ");`
`String pin = input.nextLine();`

`Rekening rekeningBaru = new Rekening(no, nama, saldo, pin);`

Bagian ini digunakan untuk meminta PIN saat pengguna membuat rekening baru. PIN kemudian dikirimkan ke constructor `Rekening` bersama dengan nomor rekening, nama pemilik, dan saldo awal.

- **Menu Tarik Tunai dengan Autentikasi**

- `System.out.print("Masukkan PIN: ");`
`String pinTarik = input.nextLine();`

```
if (akunAktif.otentikasi(pinTarik)) { System.out.print("Masukkan
nominal tarik tunai: ");
double tarik = input.nextDouble();
akunAktif.tarikTunai(tarik);
} else {
System.out.println( "Akses Ditolak: PIN yang Anda masukkan
salah!");
}
```

Bagian ini digunakan untuk memberikan keamanan pada proses tarik tunai. Sistem meminta PIN terlebih dahulu dan memanggil method `otentikasi()`. Jika PIN benar, proses tarik tunai dilanjutkan. Jika PIN salah, transaksi dibatalkan dan sistem menampilkan pesan penolakan akses.

- **Menu Total Transaksi dengan Autentikasi**

- `System.out.print("Masukkan PIN: ");`

```
String pinMutasi = input.nextLine();

if (akunAktif.otentikasi(pinMutasi)) {
    akunAktif.cetakTotalTransaksi();
} else {
    System.out.println( "Akses Ditolak: PIN yang Anda masukkan salah!" );
}
```

Bagian ini memberikan autentikasi sebelum pengguna dapat melihat total transaksi dan riwayat transaksi. Jika PIN yang dimasukkan benar, method cetakTotalTransaksi() dijalankan. Jika salah, akses ditolak.

- **Menutup Program**

→ isRunning = false;

Digunakan untuk menghentikan perulangan menu sehingga program dapat berakhir ketika pengguna memilih menu Keluar.

2.3 Transaksi.java

```
package MiniBank.copy.copy;

public class Transaksi {
    // 1. Mengubah semua atribut menjadi private
    private String idTransaksi;
    private String jenis;
    private double nominal;

    // Constructor
    public Transaksi(String id, String jenis, double nominal) {
        this.idTransaksi = id;
        this.jenis = jenis;
        this.nominal = nominal;
    }

    // 2. Hanya menyediakan Getter (Read-Only)
    public String getIdTransaksi() { return idTransaksi; }
    public String getJenis() { return jenis; }
    public double getNominal() { return nominal; }

    public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis +
            " | Nominal: Rp" + nominal);
    }
}
```

```
}
```

- **Atribut Transaksi**

- `private String idTransaksi;`
`private String jenis;`
`private double nominal;`

Atribut digunakan untuk menyimpan identitas transaksi, jenis transaksi, dan jumlah uang yang ditransaksikan. Atribut dibuat `private` sebagai penerapan enkapsulasi.

- **Constructor Transaksi**

- `public Transaksi(String id, String jenis, double nominal)`

Constructor digunakan untuk memberikan nilai awal pada objek transaksi ketika objek tersebut dibuat.

- **Getter**

- `public String getIdTransaksi()`
`public String getJenis()`
`public double getNominal()`

Getter digunakan untuk membaca nilai atribut transaksi dari class lain tanpa memberikan akses langsung terhadap atribut yang bersifat `private`.

- **Method Cetak Detail**

→ `public void cetakDetail()`

Method ini digunakan untuk menampilkan ID, jenis, dan nominal dari suatu transaksi. Method tersebut dipanggil ketika sistem menampilkan riwayat transaksi.

2.4 Tugas 1: Mengamankan CLI dengan Otentikasi

```
package MiniBank.copy.copy;

import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
```

```

Rekening akunAktif = null;
boolean isRunning = true;
System.out.println("=== SISTEM PERBANKAN MINI ===");
while (isRunning) {

    System.out.println("\nMenu Utama");
    System.out.println("1. Buka Rekening Baru");
    System.out.println("2. Setor Tunai");
    System.out.println("3. Tarik Tunai");
    System.out.println("4. Cek Informasi Rekening");
    System.out.println("5. Ganti Akun");
    System.out.println("6. Total Transaksi");
    System.out.println("7. Keluar");
    System.out.print("Pilih Menu: ");

    int pilihan = input.nextInt();
    input.nextLine();

    switch (pilihan) {
    case 1:
        System.out.print("Masukkan No Rekening: ");
        String no = input.nextLine();

        System.out.print("Masukkan Nama Pemilik: ");
        String nama = input.nextLine();

        System.out.print("Masukkan Saldo Awal: ");
        double saldo = input.nextDouble();
        input.nextLine(); // membersihkan enter

        System.out.print("Masukkan PIN (6 digit): ");
        String pin = input.nextLine();

        Rekening rekeningBaru = new Rekening(no, nama, saldo,
pin);

        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;

        System.out.println("Rekening berhasil ditambahkan.");
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;
    }
}

```

```

        case 3:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum memiliki
rekening!");
            } else {
                System.out.print("Masukkan PIN: ");
                String pinTarik = input.nextLine();

                // Verifikasi PIN sebelum tarik tunai
                if (akunAktif.otentikasi(pinTarik)) {
                    System.out.print("Masukkan nominal tarik
tunai: ");

                    double tarik = input.nextDouble();
                    akunAktif.tarikTunai(tarik);
                } else {
                    System.out.println(
                        "Akses Ditolak: PIN yang Anda masukkan
salah!"
                    );
                }
            }
            break;

        case 4:
            if (akunAktif == null) {
                System.out.println("Error: Anda belum membuka
rekening!");
            } else {
                akunAktif.cekInformasi();
            }
            break;

        case 5:
            if (daftarRekening.isEmpty()) {
                System.out.println("Belum ada rekening yang
tersedia.");
            } else {
                System.out.print(
                    "Masukkan nomor rekening: "
                );
                String nomorCari = input.nextLine();
                boolean ditemukan = false;
                for (Rekening rekening : daftarRekening) {
                    if
(rekening.getNomorRekening().equals(nomorCari)) {
                        akunAktif = rekening;
                        ditemukan = true;
                        System.out.println("Akun berhasil diganti
ke " + rekening.getNamaPemilik());
                        break;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    if (!ditemukan) {
        System.out.println("Rekening tidak
ditemukan.");
    }
    }
    break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan PIN: ");
            String pinMutasi = input.nextLine();

            // Verifikasi PIN sebelum cetak mutasi
            if (akunAktif.otentikasi(pinMutasi)) {
                akunAktif.cetakTotalTransaksi();
            } else {
                System.out.println(
                    "Akses Ditolak: PIN yang Anda masukkan
salah!"
                );
            }
        }
        break;

    case 7:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
}
input.close();
}

}

package MiniBank.copy.copy;

import java.util.ArrayList;
import java.text.NumberFormat;
import java.util.Locale;

public class Rekening {
    // 1. Mengunci atribut dengan 'private'
    private String nomorRekening;

```

```

private String namaPemilik;
private double saldo;
private String pin; // Data sensitif

    NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

    // Implementasi Asosiasi (1-to-many)
    ArrayList<Transaksi> riwayatTransaksi;

    // 2. Modifikasi Constructor untuk menerima PIN awal
    public Rekening(String nomor, String nama, double saldoAwal, String
pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        // Validasi PIN di dalam Constructor
        if (pinAwal.length() == 6) {
            this.pin = pinAwal;
        } else {
            System.out.println("Peringatan: PIN harus 6 digit! Menggunakan
PIN default 123456");
            this.pin = "123456";
        }

        this.riwayatTransaksi = new ArrayList<>();
        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat.");
        // Wajib menginstalasi ArrayList didalam constructor agar tidak
NullPointerException
        this.riwayatTransaksi = new ArrayList<>();

        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + saldo);
    }

    // 3. Getter untuk atribut yang diizinkan dibaca publik
    public String getNomorRekening() { return nomorRekening; }
    public String getNamaPemilik() { return namaPemilik; }

    //4. Method Otentikasi Internal (Validasi Enkapsulasi)
    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            // Merekam riwayat (Pembuatan objek transaksi di dalam
method)
            String idTrx = "TRX-S" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);
            riwayatTransaksi.add(trxBaru);

```

```

        System.out.println("Setor tunai Rp" + nominal+ " berhasil.
Saldo saat ini: Rp" + saldo);
    } else {
        System.out.println("Gagal: Nominal setor harus lebih dari
0!");
    }
}

public void cekInformasi() {

    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening : " + nomorRekening);
    System.out.println("Nama Pemilik : " + namaPemilik);
    System.out.println("Saldo Akhir : " + saldo);
}

// Method tarik tunai
public void tarikTunai(double nominal) {
    if (nominal < 10000) {
        System.out.println(
            "Transaksi Gagal: Minimal nominal penarikan Rp10000");
    } else if (nominal > saldo) {
        System.out.println("Transaksi Gagal: Saldo tidak mencukupi.
Saldo Anda: Rp" + saldo);
    } else {
        saldo -= nominal;
        // Merekam riwayat penarikan
        String idTrx = "TRX-T-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
        riwayatTransaksi.add(trxBaru);

        System.out.println("Penarikan berhasil.");
        System.out.println("Jumlah penarikan: Rp" + nominal);
        System.out.println("Saldo sekarang: Rp" + saldo);
    }
}

public void cekMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening
ini");
    } else {
        int mulai = Math.max(0, riwayatTransaksi.size() -
3);
        System.out.println("--- TRANSAKSI TERBARU ---");
        for (int i = mulai; i < riwayatTransaksi.size();
i++) {
            riwayatTransaksi.get(i).cetakDetail();
        }
    }
}

public void cetakTotalTransaksi() {

```

```

double totalSetor = 0;
double totalTarik = 0;

for (Transaksi trx : riwayatTransaksi) {

    if (trx.getJenis().equals("Kredit")) {
        totalSetor += trx.getNominal();
    } else if (trx.getJenis().equals("Debit")) {
        totalTarik += trx.getNominal();
    }
}

NumberFormat rupiah = NumberFormat.getCurrencyInstance(new
Locale("id", "ID"));

System.out.println("--- TOTAL TRANSAKSI ---");
System.out.println("Total Setor : " + rupiah.format(totalSetor));
System.out.println("Total Tarik : " + rupiah.format(totalTarik));

System.out.println("\n--- 3 TRANSAKSI TERBARU ---");

int mulai = Math.max(0, riwayatTransaksi.size() - 3);

for (int i = mulai; i < riwayatTransaksi.size(); i++) {
    riwayatTransaksi.get(i).cetakDetail();
}

}
}

```

○ Modifikasi Class Rekening.java

- **tribut Privat pin:** Atribut PIN disembunyikan menggunakan *access modifier* private untuk menerapkan prinsip enkapsulasi agar tidak dapat diakses atau diubah langsung dari luar class.
- **Constructor Update:** Constructor diperbarui agar menerima parameter pinInput. Terdapat pula pengecekan kondisi (pinInput.length() == 6) untuk memastikan variabel pin terisi dan valid, sekaligus menghindari *NullPointerException*.
- **Method otentikasi(String inputPin):** Method ini mengembalikan nilai boolean (true jika PIN cocok, dan false jika tidak). Method inilah yang digunakan oleh class Main untuk memvalidasi transaksi.

- **Modifikasi Classs Main.java**

- **Menu 1 (Buka Rekening):** Sebelum membuat objek rekeningBaru, pengguna diminta menginputkan PIN. Nilai PIN ini diteruskan ke *constructor* class Rekening.

- **Menu 3 (Tarik Tunai):** Sebelum meminta nominal penarikan, program mewajibkan pengguna memasukkan PIN. Program mengevaluasi hasil pemanggilan akunAktif.otentikasi(pinInput). Jika bernilai true, transaksi diproses; jika false, program menampilkan pesan penolakan.

- **Menu 6 (Total Transaksi / Cetak Mutasi):** Sebelum menampilkan data riwayat transaksi, sistem meminta PIN pengguna. Pemanggilan method cetakTotalTransaksi() hanya dieksekusi apabila pemanggilan otentikasi(pinInput) mengembalikan nilai true.

Output:

```
Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Keluar
Pilih Menu: 1
Masukkan No Rekening: 123
Masukkan Nama Pemilik: rara
Masukkan Saldo Awal: 100000
Buat PIN Keamanan (6 digit): 111111
Rekening atas nama rara berhasil dibuat dengan saldo Rp100000.0 dan PIN telah terdaftar.
```

```
Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Keluar
Pilih Menu: 6
Masukkan PIN Anda untuk verifikasi: 111111
|--- TOTAL TRANSAKSI ---
Total Setor : Rp300.000,00
Total Tarik : Rp90.000,00

--- 3 TRANSAKSI TERBARU ---
ID: TRX-S1790545615441 | Jenis: Kredit | Nominal: Rp300000.0
ID: TRX-T-1790545628671 | Jenis: Debit | Nominal: Rp90000.0
```

```
Menu Utama
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
6. Total Transaksi
7. Keluar
Pilih Menu: 3
Masukkan PIN Anda untuk verifikasi: 111111
Masukkan nominal tarik tunai: 90000
|Penarikan berhasil.
Jumlah penarikan: Rp90000.0
Saldo sekarang: Rp310000.0
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum dan penyelesaian tugas yang telah dilakukan, dapat disimpulkan bahwa penerapan fitur keamanan berupa otentikasi PIN berhasil meningkatkan aspek enkapsulasi data serta keamanan transaksi pada program *Mini Bank*. Penggunaan method otentikasi(String inputPin) memungkinkan sistem melakukan validasi identitas pengguna secara fleksibel tanpa mengekspos atribut pin yang bersifat privat keluar dari class Rekening. Selain itu, pengintegrasian logika percabangan kondisi pada class Main memastikan bahwa eksekusi transaksi sensitif, seperti penarikan tunai dan pencetakan mutasi, hanya dapat dilakukan jika hasil otentikasi bernilai *true*. Dengan demikian, seluruh konsep Pemrograman Berorientasi Objek mulai dari pembentukan class, object, constructor, method, manipulasi state, hingga enkapsulasi data telah berhasil diterapkan dengan baik dalam suatu alur program perbankan yang aman.